

Widget de Chat Embebible

Widget de Chat Embebible — Guía técnica y de integración

Chatbot flotante, embebible y escalable para cualquier agente IA de FastChat DJ. Primer caso de uso: el **cotizador médico Vida Buena**, con el chat integrado en la misma página tomando el hilo de la conversación. Fecha: 2026-07-18.

1. Qué es

Un **widget de chat** que cualquier página web puede mostrar (una burbuja flotante abajo a la derecha) para conversar con **su** agente IA. La conversación:

- **Mantiene el hilo** entre mensajes y entre recargas de página (`session_id` persistente en `localStorage`), igual que un chat real multi-turno.
- Usa la **misma cadena** que el resto del sistema: RAG por-agente (Weaviate), memoria, y el proveedor LLM configurado (Gemini / OpenAI / Claude / Ollama).
- Renderiza **markdown** (negritas, listas, enlaces, código) en las respuestas.

Se apoya en la API pública ya existente (`/api/ia/consultar/`) pero **sin exponer ninguna credencial en el navegador**.

2. Arquitectura y seguridad

2.1. El problema que resuelve el diseño

La API pública se autentica con `Authorization: Bearer <webservice_token>` . Ese token es **secreto**: quien lo tenga puede consumir el agente (y gastar tokens del LLM). Si el widget lo pusiera en el JavaScript del navegador, cualquiera podría leerlo desde el código fuente y abusarlo.

2.2. La solución: embed key firmado + proxy server-side

Navegador (página del cliente)

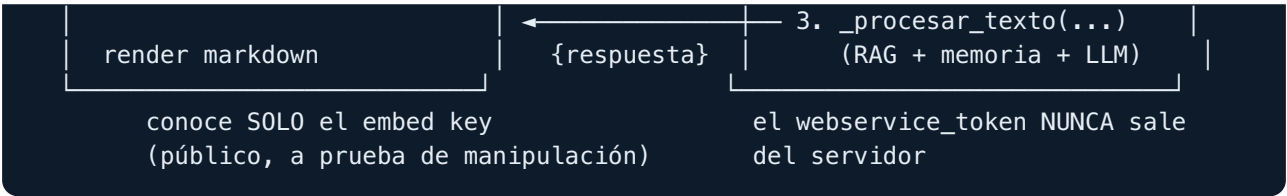
```
<script embed.js>
data-embed-key="eyJ..."
```

burbuja de chat

POST {key,
mensaje,
session_id}

Servidor FastChat DJ

```
/chat-widget/api/mensaje/  
1. verifica firma del key  
2. resuelve agente + su  
ApiKeyIA (server-side)
```



- El **embed key** es un token **firmado** con `django.core.signing` (clave `SECRET_KEY` del servidor). Solo contiene el `id` del agente (y, opcional, los dominios permitidos). Es **público** y a **prueba de manipulación**: el cliente no puede cambiarlo para apuntar a otro agente ni escalar privilegios; cualquier alteración invalida la firma → **403**.
- El **proxy** (`crm/chat_widget.py::widget_mensaje_view`) valida la firma, resuelve el agente y su `ApiKeyIA` en el servidor, y reutiliza exactamente `crm.api_ia._procesar_texto` (la misma función que la API pública). **No hay lógica duplicada** y el `webservice_token` / la key del proveedor nunca viajan al navegador.

2.3. Controles de seguridad

Control	Cómo
Anti-manipulación	Embed key firmado (HMAC vía <code>SECRET_KEY</code>). Alterarlo → 403.
Sin credenciales en cliente	El navegador solo maneja el embed key público.
Rate limiting	<code>@rate_limit(limit=40, seconds=60)</code> por IP en el proxy.
Restricción por dominio (opcional)	Se hornean dominios permitidos dentro del embed key; el proxy valida <code>Origin</code> / <code>Referer</code> .
Alertas de consumo	Reusa <code>verificar_alerta_consumo</code> (mismo control de la API).
CORS	El proxy responde <code>Access-Control-Allow-Origin</code> y maneja <code>OPTIONS</code> (preflight).

Sin BD, sin migración. El diseño no crea tablas nuevas: aprovecha los `AgentesIA` + `ApiKeyIA` existentes y firma un token. Nada que migrar.

3. Rutas

Método	Ruta	Descripción
GET		

Método	Ruta	Descripción
	/chat-widget/ embed.js	El JS del widget (autocontenido, CORS *, cache 5 min).
POST	/chat-widget/api/ mensaje/	Proxy de mensajes. Body: {embed_key, mensaje, session_id}.
OPTIONS	/chat-widget/api/ mensaje/	Preflight CORS (204).
GET	/chat/ <embed_key>/	Página de chat autónoma (una por cliente).

Respuesta del proxy:

```
{ "ok": true, "respuesta": "...", "session_id": "cw-abc123", "tokens":
  {"entrada":..., "salida":..., "total":...} }
```

4. Integración

4.1. Opción A — Burbuja embebida en una página existente

Pega **un solo** `<script>` antes de `</body>`:

```
<script src="https://TU-DOMINIO/chat-widget/embed.js"
  data-embed-key="eyJhIjoyMn0:...."
  data-titulo="Asesor Vida Buena"
  data-color="#1b6ec2"
  data-bienvenida="¡Hola! ¿En qué puedo ayudarte?"></script>
```

Atributos `data-*`:

Atributo	Obligatorio	Descripción
data-embed-key	sí	Embed key del agente (ver §5).
data-titulo	no	Título en la cabecera del chat.
data-color	no	Color principal (hex). Por defecto <code>#1b6ec2</code> .
data-bienvenida	no	Primer mensaje del asistente al abrir.
data-abierto	no	<code>"true"</code> para abrir el panel automáticamente.

4.2. Opción B — Página de chat autónoma (una por cliente)

Comparte directamente la URL:

```
https://TU-DOMINIO/chat/<embed_key>/
```

Es una landing a pantalla completa, con la marca del agente y el chat abierto. Ideal para clientes que quieren **su propia página** de atención sin montar nada.

4.3. API JavaScript (control desde la página host)

El widget expone `window.VidaChat` :

```
VidaChat.abrir();           // abre el panel
VidaChat.cerrar();         // lo cierra
VidaChat.enviar("Hola");    // envía un mensaje mostrándolo en el chat
VidaChat.setPrefacio("Contexto..."); // antepone contexto (oculto) al PRÓXIMO mensaje
VidaChat.nuevaSesion();     // reinicia el hilo (nueva session_id)
```

`setPrefacio` sirve para que la página host inyecte contexto al agente. Ej: en el cotizador, tras generar una cotización se puede hacer `VidaChat.setPrefacio("El usuario acaba de cotizar el plan Magno 30.000 para 28 años.")` para que el chatbot continúe el hilo con ese dato.

5. Escalabilidad multi-cliente — “cada uno su página con su API”

El modelo escala sin código nuevo por cliente:

1. Cada cliente tiene su **AgentesIA** + su **ApiKeyIA** (su proveedor, su modelo, su base de conocimiento RAG). Eso ya existe en el panel.
2. Se genera **su embed key** con el comando de onboarding:

```
# Onboarding básico
python manage.py generar_embed_widget --agente-id 22 --base https://TU-DOMINIO

# Restringido al dominio del cliente (recomendado en producción)
python manage.py generar_embed_widget --agente-id 22 \
    --origins https://cliente.com https://www.cliente.com \
    --base https://TU-DOMINIO
```

El comando imprime: el **embed key**, el **snippet** `<script>` listo para pegar, y la **URL de la página autónoma**. No toca la BD ni expone secretos.

1. El cliente pega el snippet en su web (o usa su página autónoma). Su widget habla con **su** agente, con **su** proveedor/API y **su** conocimiento.

Selección automática del agente en el cotizador

`cotizador_view` elige el agente del chat así:

1. `?agente_id=<id>` explícito en la URL (para montar cotizadores de otros clientes apuntando a su agente).

2. Por defecto: el agente de la empresa que tenga una **herramienta cotizar*** (el que sabe cotizar). Si no hay, el primer agente activo de la empresa.

6. Integración concreta en el cotizador Vida Buena

- `cotizador/views.py::cotizador_view` resuelve el agente (el que tiene la tool `cotizar_vida_buena`, id 22) y pasa `chat_embed_key` al template.
- `cotizador/templates/cotizador/cotizador.html` incluye el `<script>` del widget solo si hay `chat_embed_key`.
- La empresa por defecto del cotizador (`_empresa_default_id`) prioriza una empresa que tenga **planes y agente**, para que cotizador + chatbot queden ambos operativos (el precio no cambia: los planes son idénticos entre empresas).

6.1. Venta asistida al elegir un plan (RAG + IA proactiva)

Cuando el usuario pulsa “**Elegir este plan**”, el cotizador no solo abre el chat: inyecta el **contexto de la cotización en pantalla** (perfil, y las primas exactas ya calculadas de todos los planes) como *prefacio oculto* y envía un mensaje. El asesor entonces, de forma proactiva:

1. Presenta los **beneficios y coberturas clave** del plan elegido (desde el RAG).
2. Lo **compara con un plan de perfil opuesto** (más económico o más completo), explicando la diferencia principal.
3. Pregunta si el usuario **tiene dudas** o quiere la cotización oficial.

Como ya recibe las primas exactas en el contexto, **no vuelve a pedir la cédula** solo para cotizar. La lógica vive en `cotizar()` (guarda `window._cotizacion`) y `elegirPlan(idx)` (arma el prefacio y llama `VidaChat.setPrefacio()` + `VidaChat.enviar()`). Es el patrón recomendado para cualquier página que quiera convertir un evento (elegir, agregar al carrito, etc.) en una conversación con contexto.

7. Verificación (producción, 2026-07-18)

- `GET /chat-widget/embed.js` → **200** (~8.4 KB, `application/javascript`).
 - Cotizador incluye el `<script>` con `data-embed-key = {"a":22}` (Vida Buena).
 - `POST /chat-widget/api/mensaje/` (agente 22, Ollama `gemma4:31b`):
 - “¿Qué planes ofrecen?” → responde como *Camila*, asesora Vida Buena ✓
 - Seguimiento misma sesión: “¿el de mayor cobertura?” → “MAGNO 30.000” (RAG + memoria) ✓
 - Embed key manipulado → **403**. Preflight `OPTIONS` → **204**.
 - Verificación visual: burbuja flotante en el cotizador; respuesta con markdown (negritas, viñetas) y coberturas reales del RAG; página autónoma `/chat/<key>/` renderiza a pantalla completa con el chat abierto.
-

8bis. Captura de leads al panel (interoperabilidad con el CRM)

Cuando el cliente conversa en el chat del cotizador y **deja su correo** (o teléfono), el lead aterriza automáticamente en el MISMO panel que los leads de WhatsApp: como **Contacto** y como **tarjeta en el Pipeline de ventas** (etapa “Nuevo Lead”), para que el equipo le dé seguimiento.

Disparador: el proxy (`widget_mensaje_view`) detecta un email/teléfono en el mensaje del usuario. El widget adjunta en cada POST un `lead_context` (perfil + plan de interés + valor estimado) que el cotizador setea al elegir un plan (`VidaChat.setLeadContext(...)`). Así el lead se crea con el contexto completo solo cuando hay datos de contacto reales (sin ruido de simples visitas).

Sin migración, reusando la infraestructura (`crm/lead_panel.py::registrar_lead`): - Se usa una **sesión web dedicada por empresa** (`web-cotizador-emp<id>`): `proveedor='meta' + estado='conectado'` → el cron de reconexión (solo toca baileys desconectado/error) NUNCA la toca; `usuario` = dueño de la empresa (`PerfilNegocioIA.usuario`) → los contactos/tarjetas aparecen en SU panel. - La conversación se crea con expiración lejana → el cron de despedida no intenta enviar WhatsApp a un lead web. - Los datos del lead (email, cédula, edad, género, plan, `origen=cotizador_web`) se guardan en `PerfilContacto.intereses_json` (campo libre, sin migración). - La tarjeta se crea en el pipeline por defecto, primera etapa, con `valor_estimado` = prima mensual × 12, y una nota con el detalle del lead. - **Idempotente por `session_id`**: reingresos del mismo chat actualizan el mismo contacto/tarjeta, no duplican.

Verificado en producción: elegir plan → chat → “mi correo es ...” → aparece la tarjeta “HECTOR AARON LLERENA AGUILERA · USD 629,00 · Origen: Cotizador web” en la columna “Nuevo Lead” del pipeline “Asistencia Médica - Proceso de Venta”.

Pendiente (arista aparte): el envío de la **cotización oficial de MGA** por la tool `cotizar_vida_buena` — su webhook tiene una URL placeholder inválida; queda a la espera del endpoint real de MGA. NO afecta la captura al panel (son independientes).

8ter. Nota de caché del embed.js

`/chat-widget/embed.js` se sirve con `Cache-Control: public, max-age=300`. Tras cambiar el JS del widget, los navegadores pueden servir la versión anterior hasta 5 minutos (o un hard-refresh). Para forzar propagación inmediata, versiona el `<script src=“.../embed.js?v=N”>` y sube N.

9. Archivos

Archivo	Rol
<code>crm/chat_widget.py</code>	Embed key firmado, proxy de mensajes (+captura de lead), JS del widget, página autónoma.
<code>crm/lead_panel.py</code>	

Archivo	Rol
	Captura de leads del cotizador al panel (Contacto + Pipeline), sin migración.
<code>crm/templates/crm/chat_widget_pagina.html</code>	Página de chat autónoma.
<code>static/css/crm/chat_widget_pagina.css</code>	Estilos de la página autónoma.
<code>crm/management/commands/generar_embed_widget.py</code>	Onboarding: genera key + snippet + URL.
<code>fastchatdj/urls.py</code>	Monta las rutas <code>/chat-widget/*</code> y <code>/chat/<key>/</code> .
<code>cotizador/views.py</code>	Resuelve el agente y pasa el embed key al cotizador.
<code>cotizador/templates/cotizador/cotizador.html</code>	Incluye el <code><script></code> del widget.